Aprendizaje por refuerzo con verificación certificada como única recompensa: cotas de GD en Isabelle (V_I, teorema)

Informe de corrida del proyecto `rl_metodos`

Carlos Manuel Orrego Franco
Universidad Nacional de Colombia, Sede Manizales

Septiembre de 2026 — informe de las corridas `20260924-062122-gpt-6-luna-high` a `20260924-091710-gpt-6-luna-high`

1 Propósito

Esta nota fija el lenguaje común de los subproyectos de `rl_metodos`. Todos comparten una misma estructura: un *proponente* produce un objeto de prueba y un *juez* independiente decide si lo acepta. La recompensa proviene únicamente del juez; el proponente nunca se evalúa a sí mismo. Lo que varía entre subproyectos es el juez, y con él el residuo de confianza que la aceptación deja sin certificar. Esa variación es deliberada: es el experimento.

Este informe agrupa 4 corridas del proponente `gpt-6-luna` (high) bajo V_I (variante de teorema), una por objetivo: (1) `gd_n1`, `20260924-062122-gpt-6-luna-high`, terminada; 3 episodios completos; (2) `gd_const_n2`, `20260924-073842-gpt-6-luna-high`, terminada; 3 episodios completos; (3) `gd_const_textbook`, `20260924-082858-gpt-6-luna-high`, terminada; 3 episodios completos; (4) `gd_const_dt`, `20260924-091710-gpt-6-luna-high`, terminada; 3 episodios completos. Presupuesto $B = 4$ intentos por episodio. Memoria entre episodios: sí (`run`; encabezado de la biblioteca en su redacción v2). Pregunta que responde: ¿escribe `gpt-6-luna` una demostración Isar de cada cota que el kernel acepte? El proponente demuestra; Isabelle verifica.

Nota de la carpeta `20260924-062122-gpt-6-luna-high` (`LEEME.md`): «# Luna, gd_n1: corrida con el analizador de respuestas anterior Esta corrida empezó el 24 sep 2026 a las 06:21, antes de que se corrigiera 'TheoremVerifier.parse' (06:50), y cada proceso carga el código al empezar. Con el analizador anterior, una respuesta con 'LEMMAS: <<<>>>' (dos '>' en vez de tres, para decir «sin lemas») hacía que el bloque de lemas se comiera la demostración: el intento salía INVALID_PROPOSAL con el motivo «el bloque de lemas debe empezar con 'lemma'», y la demostración nunca llegaba a Isabelle. Le pasó a Luna en 4 de sus 12 intentos. Los otros 8 son PROOF_FAILED genuinos (Isabelle rechazó la demostración). Astra corre gd con el analizador corregido; en gd_n1 la compa...»

Este informe sigue la estructura de la nota técnica: el MDP común (§2), la interfaz de verificador y sus tres instancias (§3), las funciones de recompensa (§4), las métricas (§5), la jerarquía de confianza entre jueces (§6) y los entornos (§7, Tabla 6); cierra con los experimentos realizados y previstos (§8), y reporta en §5 los valores medidos.

2 El MDP común

Definición 1 (Entorno de demostración verificada)**.** *Un entorno es una tupla* $\mathcal{E} = (T, \mathcal{A}, V, B)$ *donde* T *es un* objetivo *inmutable (un enunciado y su contexto),* $\mathcal{A}$ *es un espacio de* propuestas*,* V *es un* verificador *y* $B \in \mathbb{N}$ *es un presupuesto de pasos. El MDP asociado es* $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ *con:*

- ***Estado*** $s_t = (T, \mathcal{C}_t, h_t, M_t)$*: el objetivo, el* contexto observable $\mathcal{C}_t$ *que el juez devuelve tras el último intento (lemas disponibles, obligaciones abiertas con su texto literal, o el valor certificado vigente), y el historial* h_t *de resultados anteriores.*

- ***Memoria*** M_t*: biblioteca de lemas que el agente demostró y que Isabelle verificó (se añaden solos a cada intento) y resumen del mejor intento de cada episodio anterior. Persiste entre episodios de la corrida. El modelo está congelado: la memoria es contexto, no entrenamiento.*

- ***Acción*** $a_t \in \mathcal{A}$*: una propuesta completa (una cadena de lemas, una anotación de invariantes y variantes, o un programa de tamaños de paso). El proponente es un modelo de lenguaje; la acción es su salida tras una extracción determinista.*

- ***Transición*** $P(s_{t+1} \mid s_t, a_t)$*: determinista dado el juez;* s_{t+1} *incorpora el resultado* $V(T, a_t)$ *y actualiza* $\mathcal{C}_{t+1}$ *y* h_{t+1}*. La estocasticidad del sistema vive en el proponente, no en la transición.*

- ***Recompensa*** $R(s_t, a_t) = r(V(T, a_t))$*, función únicamente del resultado del juez (§4).*

- ***Terminación****: el episodio termina en el primer* t *con resultado* Accepted*, o cuando* $t = B$*.*

Observación 1 (El objetivo es inmutable)**.** *T no forma parte del espacio de acciones. Cualquier propuesta que modifique el enunciado, sus hipótesis o su tipo se rechaza con etiqueta* Statement_Drift *(§4.3). Esta restricción se impone en código mediante un hash del enunciado congelado, no por convención. Es la* frontera semántica *del proyecto: el lugar donde la confianza no puede delegarse al juez. En este informe, los hashes de los objetivos son:* `gd_n1: 469e6d77bc588f4a; gd_const_n2: d0639dda521234f4; gd_const_textbook: f4147ea80e7e6c69; gd_const_dt: fe2f1ffb6b45a25f`*.*

3 La interfaz de verificador y sus tres instancias

Definición 2 (Verificador)**.** *Un verificador es una función* $V : \mathcal{T} \times \mathcal{A} \to \mathcal{O}$ *que devuelve un* resultado

$$o = (\texttt{accepted},\ \texttt{label},\ \texttt{partial},\ \texttt{cost_s},\ \texttt{raw}),$$

con $\texttt{accepted} \in \{0, 1\}$*,* $\texttt{label} \in L$ *y* $\texttt{partial} \in [0, 1]$*, donde* L *es un conjunto finito de etiquetas de resultado propio de cada juez,* `partial` *es una medida de progreso que sólo se usa bajo recompensa moldeada, y* `raw` *es la salida íntegra del juez para auditoría.*

3.1 Juez de grafo de lemas (V_G)

Proviene de los subproyectos 01–05. Un objetivo T es un grafo dirigido acíclico $G = (\Lambda, D)$ de pasos $\lambda \in \Lambda$, cada uno con dependencias $D(\lambda) \subseteq \Lambda$, un subconjunto $\Delta \subset \Lambda$ de *distractores* (pasos falsos o inaplicables, tomados de errores reales de estudiantes) y un paso distinguido Qed. Una propuesta es una secuencia $a = (\lambda_1, \ldots, \lambda_k)$ de identificadores anónimos (`L01`, `L02`, ..., barajados con semilla fija); no se muestran los títulos de los pasos ni las refutaciones del profesor, que delatan al distractor. El juez acepta si existe $j \le k$ con $\lambda_j =$ Qed tal que, para todo $i \le j$,

$$\lambda_i \notin \Delta, \quad \lambda_i \notin \{\lambda_1, \ldots, \lambda_{i-1}\}, \quad D(\lambda_i) \subseteq \{\lambda_1, \ldots, \lambda_{i-1}\};$$

lo que venga después de Qed se ignora. Etiquetas, asignadas en el primer paso que falla: Accepted (la cadena llega a Qed), Invalid_Step (dependencia no establecida o paso repetido), Distractor (el paso es un distractor), Unknown_Lemma (identificador ajeno al grafo) y Incomplete (todos los pasos son válidos pero la cadena no llega a Qed; es un resultado por intento, no por presupuesto). Progreso parcial: fracción del cierre de dependencias de Qed establecida antes del primer fallo.

3.2 Juez PEP (V_P)

Proviene del subproyecto 06, que se usa sin modificaciones. El objetivo es una instancia: la clase de funciones L-suaves (μ-fuertemente) convexas con $L = 1$, un horizonte n, un criterio ($f(x_n) - f^*$ o $\|\nabla f(x_n)\|^2$, con $\|x_0 - x^*\| \leq 1$) y si se admiten pasos negativos; una propuesta es un programa `schedule(n)` que genera pasos $(h_0, \ldots, h_{n-1})$. El programa se ejecuta en un intérprete aparte (en macOS, bajo un *sandbox* sin red ni escritura en disco) y el juez puntúa, en un proceso que el programa no tocó, la sucesión literal que devuelve; las referencias se leen una sola vez al empezar. El juez resuelve el problema de estimación de desempeño (PEP) como un programa semidefinido y obtiene el peor caso $\tau(h)$; el certificado dual es la demostración de la cota (Drori and Teboulle, 2014; Taylor et al., 2017). En el modo por defecto, `improve`, toda mejora afirmada se recalcula con tolerancias finas y se certifica en aritmética racional exacta; se acepta sólo si la cota certificada es estrictamente menor que τ_{ref}. La referencia τ_{ref} es el mínimo entre la tabla publicada de óptimos numéricos, que sólo cubre $f(x_n) - f^*$ con $\mu = 0$ (Das Gupta et al., 2024; Grimmer et al., 2024), y la búsqueda propia de 06 por entropía cruzada; en las variantes abiertas (norma del gradiente, fuertemente convexo, pasos negativos) sólo hay búsqueda propia. Etiquetas: CERTIFIED (mejora estricta de τ_{ref} con certificado racional exacto; es la aceptación), MATCHED (sólo en modo `match`: iguala τ_{ref} con tolerancia relativa 10^{-4}, sin certificado exacto), INVALID_SCHEDULE (el programa falla, se agota su tiempo o no devuelve n números finitos válidos), NOT_IMPROVED (programa válido que no mejora τ_{ref}) y CERTIFICATION_FAILED (mejora numérica cuyo certificado exacto falla).

3.3 Juez Isabelle (V_I)

Proviene del subproyecto 08 y del marco Isabelle/ITrees (Bryant et al., 2026). Un objetivo es un programa imperativo junto con un triple de Hoare de corrección total $H[P]\,C\,[Q]$, ambos congelados; una propuesta es una anotación $a = (inv, var)$ de invariante y variante de bucle y, en fase 2, además lemas auxiliares con su demostración, escritos por el agente y verificados en el kernel antes de usarse. El juez procede por etapas: (0) la anotación pasa una lista blanca de símbolos y el kernel demuestra que el programa anotado es igual al congelado con anotaciones fijas; (0b) si hay lemas, pasan un filtro léxico (sólo enunciados con su demostración: nada de `sorry`, definiciones, notación ni ML) y se compilan en el contexto de las hipótesis, comprobando que el enunciado congelado se lee igual y que no aparecen axiomas nuevos; (1) inserta la anotación y ejecuta el generador de condiciones de verificación `vcg`; (2) intenta cerrar cada obligación con automatización de una línea (`try0` y, si falla, Sledgehammer con tiempo límite fijo), y sólo cuenta una sugerencia de Sledgehammer si su reconstrucción (*preplay*) terminó; (3) si todas cierran, ensambla el teorema completo, lo compila y lo traslada al programa congelado; (4) comprueba que no dependa de oráculos, de `skip_proof` ni de axiomas nuevos. Etiquetas: ACCEPTED (todas las etapas pasan: el teorema está en el kernel), VC_OPEN (alguna obligación sin prueba de una línea), VCG_FAILED (la anotación no se analiza o no tipa, o `vcg` falla), LEMMA_FAILED (el kernel rechaza la demostración de algún lema del agente), ASSEMBLY_FAILED (las obligaciones se cierran por separado pero el teorema ensamblado no compila), UNSOUND (`sorry`, `oops`, oráculos, `skip_proof` o axiomas nuevos), STATEMENT_DRIFT (la propuesta sale de su hueco o el kernel no prueba que el programa anotado sea el congelado), TIMEOUT (se agota el tiempo de compilación) y INVALID_PROPOSAL (falta el invariante o el variante, o hay símbolos o comandos no admitidos). Progreso parcial: fracción de obligaciones cerradas en VC_OPEN y ASSEMBLY_FAILED, y 0 en las demás.

Variante de teorema. El objetivo es un teorema puro de Isabelle/HOL sobre un espacio con producto interno; la acción del agente es la demostración (texto Isar) más lemas auxiliares opcionales, y el juez sólo compila: no añade automatización. Hipótesis congeladas, con $L = 1$: $f(y) + \langle g(y), x - y\rangle + \frac{1}{2}\|g(x) - g(y)\|^2 \leq f(x)$ para todo x, y (`interp`) y $g(x_s) = 0$ (`opt`); gd g hs x_0 aplica $x_{k+1} = x_k - h_k\, g(x_k)$ con los pasos de la lista hs. El enunciado leído antes y después del texto del agente debe coincidir término a término, y el teorema se exporta al contexto global usando sólo

`interp` y `opt`. Etiquetas: ACCEPTED (compila sin oráculos, sin `skip_proof` ni axiomas nuevos y el teorema es el enunciado congelado), PROOF_FAILED (errores en la demostración del teorema), LEMMA_FAILED (errores en los lemas del agente), UNSOUND (oráculos, `skip_proof`, axiomas nuevos, o `sorry`/`oops` en el texto), STATEMENT_DRIFT (el teorema probado no es el congelado, el objetivo cambió en disco o hay comandos que cambian el significado), TIMEOUT (se agota el tiempo de compilación) y INVALID_PROPOSAL (comandos o nombres no admitidos). Enunciados de este informe:

- `gd_n1` ($n = 1$ fijo; cota exacta del paso óptimo; hash `469e6d77bc588f4a`):

$$f(\text{gd } g\ [3/2]\ x_0) - f(x_s) \le \tfrac{1}{8}\,\|x_0 - x_s\|^2$$

- `gd_const_n2` ($n = 2$ fijo; cota exacta de Drori–Teboulle; hash `d0639dda521234f4`):

$$f(\text{gd } g\ [1,1]\ x_0) - f(x_s) \le \tfrac{1}{10}\,\|x_0 - x_s\|^2$$

- `gd_const_textbook` (todo n; cota clásica; hash `f4147ea80e7e6c69`):

$$0 < n \Longrightarrow f(\text{gd } g\ (1,\ldots,1)_n\ x_0) - f(x_s) \le \frac{\|x_0 - x_s\|^2}{2n}$$

- `gd_const_dt` (todo n; cota exacta de Drori–Teboulle; hash `fe2f1ffb6b45a25f`):

$$f(\text{gd } g\ (1,\ldots,1)_n\ x_0) - f(x_s) \le \frac{\|x_0 - x_s\|^2}{4n+2}$$

Observación 2 (Existencia frente a descubrimiento)**.** *Para los objetivos con n fijo existe una demostración que V_I acepta: la que se genera desde un certificado racional exacto del PEP (multiplicadores $\lambda_{ij} \ge 0$ de las desigualdades de interpolación y una suma de cuadrados, comprobados en aritmética racional; `targets/gd_cert/`), escrita en Isar como instancias de `interp`, cuadrados no negativos y aritmética lineal. Con el juez real, con todos sus filtros: `gd_n1`: variante `C`, ACCEPTED en 5,4 s; `gd_const_n2`: variante `C`, ACCEPTED en 5,3 s (validación del `2026-09-24 06:10`, `logs/gd_validacion.json`). La misma validación comprobó que las hipótesis son satisfacibles ($f(x) = x^2/2$, $g = \mathrm{id}$, $x_s = 0$ en $\mathbb{R}$), que `sorry` lo detienen el filtro y el kernel, y que `by simp` no demuestra el objetivo de un paso. Para los objetivos "para todo n" no hay cota superior en este juez: los teoremas son ciertos (la cota de libro de texto y la de Drori and Teboulle (2014)), pero no tenemos su demostración formal. Lo que el proponente no ve es el certificado: descubrir qué desigualdades combinar, y con qué pesos, es justamente la parte difícil.*

4 Funciones de recompensa

4.1 Recompensa base (todos los jueces)

$$r(o) = \begin{cases} 1 & \text{si } o.\texttt{accepted} = 1, \\ 0 & \text{en otro caso.} \end{cases}$$

Es la única recompensa activa por defecto. Es binaria a propósito: cualquier señal intermedia introduce un juicio sobre qué es "progreso", y ese juicio debe declararse, no esconderse.

4.2 Recompensa moldeada (opcional, tras bandera)

$$r_\beta(o) = r(o) + \beta \cdot o.\texttt{partial} \cdot \mathbb{1}[o.\texttt{accepted} = 0], \qquad \beta \in [0, 0.5].$$

Para V_P se conserva la recompensa original del subproyecto 06, que es continua por naturaleza: $r_P(o) = \max\{-2,\ \log(\tau_{\text{ref}}/\tau(h))\}$ si el programa es válido y -3 en otro caso, donde τ_{ref} es la mejor cota certificada conocida. Se reporta también su binarización $\mathbb{1}[\tau(h) < \tau_{\text{ref}}]$ para hacerla comparable con los otros jueces.

4.3 Penalización por deriva del enunciado

STATEMENT_DRIFT recibe recompensa 0, no negativa, en la configuración base. La razón es metodológica: queremos *contar* cuántas veces el proponente intenta cambiar la pregunta, y una penalización alteraría la frecuencia que se pretende medir. Una variante con penalización $-\delta$ queda disponible tras bandera para experimentos de control.

En esta corrida: recompensa base binaria $r(o)$; la moldeada está desactivada ($\beta = 0$); penalización de deriva $\delta = 0$ (STATEMENT_DRIFT recibe 0). En esta corrida el proponente no aprende de la recompensa: el modelo está congelado y la recompensa se registra.

5 Métricas

Para cada entorno, juez y proponente se registran:

1. **Intentos hasta aceptación**: mediana y distribución de t^* sobre episodios, con $t^* = B$ si no hay aceptación.

   En esta corrida: (1) t^* mediana 4 sobre 3 episodios completos, censurada en $B = 4$; por episodio: ep. 1: 4 (sin aceptar), ep. 2: 4 (sin aceptar), ep. 3: 4 (sin aceptar); aceptados 0 de 3. (2) t^* mediana 4 sobre 3 episodios completos, censurada en $B = 4$; por episodio: ep. 1: 4 (sin aceptar), ep. 2: 4 (sin aceptar), ep. 3: 4 (sin aceptar); aceptados 0 de 3. (3) t^* mediana 4 sobre 3 episodios completos, censurada en $B = 4$; por episodio: ep. 1: 4 (sin aceptar), ep. 2: 4 (sin aceptar), ep. 3: 4 (sin aceptar); aceptados 0 de 3. (4) t^* mediana 4 sobre 3 episodios completos, censurada en $B = 4$; por episodio: ep. 1: 4 (sin aceptar), ep. 2: 4 (sin aceptar), ep. 3: 4 (sin aceptar); aceptados 0 de 3. En conjunto, t^* mediana 4 sobre 12 episodios completos.

2. **Distribución de etiquetas**: conteo de cada $\ell \in L$ sobre todos los intentos.

   En esta corrida: Tabla 1: 48 intentos en total.

3. **Razón de deriva**: para V_I, el cociente entre el número de resultados STATEMENT_DRIFT y el de VC_OPEN; para V_G, el cociente entre DISTRACTOR e INVALID_STEP. Mide cuán a menudo el proponente "resuelve" cambiando la pregunta o cayendo en un error de estudiante.

   En esta corrida: En la variante de teorema el análogo de VC_OPEN es PROOF_FAILED: STATEMENT_DRIFT/PROOF_FAILED $= 1/24 = 0{,}04$.

4. **Transferencia**: reducción de t^* en el objetivo T_2 cuando el proponente ha sido entrenado (o condicionado con ejemplos aceptados) sobre T_1, frente a partir de cero.

   En esta corrida: No aplica: la memoria de esta corrida recuerda entre episodios del *mismo* objetivo; no es transferencia entre objetivos, y no se midió ninguna.

5. **Comparación entre jueces**: para un mismo método y un mismo proponente, t^* y etiquetas bajo V_G frente a V_I. Es la medida directa de cuánto de la "facilidad" de un juez proviene del trabajo humano incorporado en él.

   En esta corrida: Tabla 2. Los enunciados de V_G y V_I no coinciden, así que la comparación es indirecta.

6. **Costo**: segundos de juez por intento; determina el cuello de botella real.

   En esta corrida: Segundos de juez por intento: media 27,3, mediana 5,1, máximo 309,3; segundos de modelo: media 217,4, mediana 193,7, máximo 748,5. Suma: 1312 s de juez y 10437 s de modelo. Duración desde el inicio hasta el último intento registrado: (1) 1 h 17 min; (2) 50 min 16 s; (3) 48 min 12 s; (4) 1 h 1 min.

etiqueta	(1)	(2)	(3)	(4)	total
Invalid_Proposal	4	0	0	0	4
Lemma_Failed	0	0	11	5	16
Proof_Failed	8	12	0	4	24
Statement_Drift	0	0	1	0	1
Timeout	0	0	0	3	3
intentos	12	12	12	12	48

Table 1: Etiquetas del juez sobre todos los intentos; las columnas siguen la numeración de las corridas en §1.

entorno	juez	proponente	ep.	acept.	B	t^* med.	intentos	s juez	s LLM
`gd_const_dt`*	V_I (teorema)	`gpt-6-luna` (high)	3	0	4	4	12	83,5	207,8
`gd_const_n2`*	V_I (teorema)	`gpt-6-luna` (high)	3	0	4	4	12	5,0	236,0
`gd_const_textbook`*	V_I (teorema)	`gpt-6-luna` (high)	3	0	4	4	12	9,2	173,8
`gd_n1`*	V_I (teorema)	`gpt-6-luna` (high)	3	0	4	4	12	11,6	252,1
`pep_fval_mu001_n4`	V_P	`gpt-6-astra` (high)	3	3	4	1	4	1,1	134,2
`pep_fval_mu001_n4`	V_P	`gpt-6-luna` (high)	3	0	4	4	12	0,5	78,6
`pep_fval_mu01_n5`	V_P	`gpt-6-astra` (high)	3	3	4	1	3	0,6	273,2
`pep_fval_mu01_n5`	V_P	`gpt-6-luna` (high)	3	0	4	4	12	0,5	82,1
`pep_gradnorm_n4`	V_P	`gpt-6-astra` (high)	3	3	4	1	3	0,6	55,0
`pep_gradnorm_n4`	V_P	`gpt-6-luna` (high)	3	0	4	4	12	0,5	86,3
`pep_gradnorm_n4_neg`	V_P	`gpt-6-astra` (high)	3	3	4	1	3	0,6	83,0
`pep_gradnorm_n4_neg`	V_P	`gpt-6-luna` (high)	3	0	4	4	12	0,5	98,7
`pep_gradnorm_n6`	V_P	`gpt-6-astra` (high)	3	3	4	1	3	0,6	69,5
`pep_gradnorm_n6`	V_P	`gpt-6-luna` (high)	3	0	4	4	12	0,6	212,1

Table 2: Corridas del mismo método (pasos de GD) en `runs/` (se ignoran las carpetas que empiezan por "_"), agrupadas por entorno, juez y proponente como en `python -m unirl summary`. "ep." cuenta episodios completos; los segundos son medias por intento evaluado; "intentos" no cuenta los Provider_Error (sin respuesta del modelo). *: entornos y proponente de este informe.

objetivo	aceptado	ep. acept.	t^* med.	etiquetas	lemas verif. (máx.)
`gd_n1`	no	0/3	4	Invalid_Proposal 4, Proof_Failed 8	0
`gd_const_n2`	no	0/3	4	Proof_Failed 12	0
`gd_const_textbook`	no	0/3	4	Lemma_Failed 11, Statement_Drift 1	0
`gd_const_dt`	no	0/3	4	Lemma_Failed 5, Proof_Failed 4, Timeout 3	2

Table 3: Por objetivo: episodios aceptados sobre completos, t^* mediano (censurado en B), etiquetas de todos los intentos y el mayor número de lemas verificados en un intento.

Primer error de Isabelle (o motivo del juez) en los intentos no aceptados, saneado y recortado:

- `gd_n1`, ep. 1, paso 1, Invalid_Proposal: el bloque de lemas debe empezar con 'lemma'
- `gd_n1`, ep. 1, paso 2, Proof_Failed: Undefined method: "nlinarith" (línea 21 de tu PROOF)
- `gd_n1`, ep. 1, paso 3, Proof_Failed: Failed to apply initial proof method (línea 21 de tu PROOF): using this: f xs + inner (g xs) (x0 - xs) + 1 / 2 * (nor...
- `gd_n1`, ep. 1, paso 4, Proof_Failed: Failed to apply initial proof method (línea 31 de tu PROOF): using this: f (x0 - (3 / 2) *R g x0) + inner (g (x0 - (3...
- `gd_n1`, ep. 2, paso 1, Invalid_Proposal: el bloque de lemas debe empezar con 'lemma'

objetivo	ep.	paso	etiqueta	lemas prop.	lemas verif.	s juez	s LLM
gd_n1	1	1	Invalid_Proposal	0	0	0	736
gd_n1	1	2	Proof_Failed	0	0	6	748
gd_n1	1	3	Proof_Failed	0	0	5	104
gd_n1	1	4	Proof_Failed	0	0	5	103
gd_n1	2	1	Invalid_Proposal	0	0	0	270
gd_n1	2	2	Proof_Failed	0	0	5	145
gd_n1	2	3	Proof_Failed	0	0	5	114
gd_n1	2	4	Invalid_Proposal	0	0	0	202
gd_n1	3	1	Invalid_Proposal	0	0	0	147
gd_n1	3	2	Proof_Failed	0	0	5	215
gd_n1	3	3	Proof_Failed	0	0	5	131
gd_n1	3	4	Proof_Failed	0	0	104	110
gd_const_n2	1	1	Proof_Failed	0	0	5	435
gd_const_n2	1	2	Proof_Failed	0	0	5	200
gd_const_n2	1	3	Proof_Failed	0	0	5	219
gd_const_n2	1	4	Proof_Failed	0	0	5	154
gd_const_n2	2	1	Proof_Failed	0	0	5	204
gd_const_n2	2	2	Proof_Failed	0	0	5	181
gd_const_n2	2	3	Proof_Failed	0	0	5	164
gd_const_n2	2	4	Proof_Failed	0	0	5	133
gd_const_n2	3	1	Proof_Failed	0	0	5	274
gd_const_n2	3	2	Proof_Failed	0	0	5	217
gd_const_n2	3	3	Proof_Failed	0	0	5	405
gd_const_n2	3	4	Proof_Failed	0	0	5	245
gd_const_textbook	1	1	Lemma_Failed	6	0	10	197
gd_const_textbook	1	2	Lemma_Failed	5	0	10	194
gd_const_textbook	1	3	Lemma_Failed	7	0	10	250
gd_const_textbook	1	4	Lemma_Failed	4	0	10	199
gd_const_textbook	2	1	Lemma_Failed	6	0	10	206
gd_const_textbook	2	2	Statement_Drift	7	0	0	172
gd_const_textbook	2	3	Lemma_Failed	7	0	10	127
gd_const_textbook	2	4	Lemma_Failed	7	0	10	122
gd_const_textbook	3	1	Lemma_Failed	4	0	11	257
gd_const_textbook	3	2	Lemma_Failed	5	0	11	90
gd_const_textbook	3	3	Lemma_Failed	5	0	10	123
gd_const_textbook	3	4	Lemma_Failed	6	0	10	150

Table 4: Intentos en orden. Parte 1.

objetivo	ep.	paso	etiqueta	lemas prop.	lemas verif.	s juez	s LLM
gd_const_dt	1	1	Proof_Failed	0	0	5	164
gd_const_dt	1	2	Proof_Failed	0	0	5	169
gd_const_dt	1	3	Proof_Failed	0	0	5	223
gd_const_dt	1	4	Timeout	1	0	309	188
gd_const_dt	2	1	Lemma_Failed	2	0	10	176
gd_const_dt	2	2	Lemma_Failed	2	1	10	194
gd_const_dt	2	3	Lemma_Failed	2	2	10	213
gd_const_dt	2	4	Lemma_Failed	1	2	10	143
gd_const_dt	3	1	Timeout	0	0	309	383
gd_const_dt	3	2	Timeout	0	0	309	213
gd_const_dt	3	3	Proof_Failed	0	2	10	181
gd_const_dt	3	4	Lemma_Failed	1	2	10	247

Table 5: Intentos en orden. Parte 2.

- gd_n1, ep. 2, paso 2, Proof_Failed: Failed to finish proof (línea 20 de tu PROOF): goal (1 subgoal): 1. [|f xs + (norm (g x0))2 / 2 <= f x0; g xs = 0|] =...
- gd_n1, ep. 2, paso 3, Proof_Failed: Undefined fact: "inner_neg_right" (línea 24 de tu PROOF)

- gd_n1, ep. 2, paso 4, INVALID_PROPOSAL: el bloque de lemas debe empezar con ‘lemma‘
- gd_n1, ep. 3, paso 1, INVALID_PROPOSAL: el bloque de lemas debe empezar con ‘lemma‘
- gd_n1, ep. 3, paso 2, PROOF_FAILED: Failed to finish proof (línea 20 de tu PROOF): goal (1 subgoal): 1. [|f xs + (norm (g x0))2 / 2 <= f x0; g xs = 0|] =...
- gd_n1, ep. 3, paso 3, PROOF_FAILED: Failed to finish proof (línea 23 de tu PROOF): goal (1 subgoal): 1. [|f xs + (norm (g x0))2 / 2 <= f x0; g xs = 0|] =...
- gd_n1, ep. 3, paso 4, PROOF_FAILED: exception Interrupt_Breakdown raised
- gd_const_n2, ep. 1, paso 1, PROOF_FAILED: Undefined method: "nlinarith" (línea 1 de tu PROOF)
- gd_const_n2, ep. 1, paso 2, PROOF_FAILED: Bad arguments for method "HOL.simp" (línea 1 de tu PROOF): [gd , opt]
- gd_const_n2, ep. 1, paso 3, PROOF_FAILED: Bad arguments for method "linarith" (línea 1 de tu PROOF): [interp [of xs x0] , interp [of x0 xs] , interp [of...
- gd_const_n2, ep. 1, paso 4, PROOF_FAILED: Outer syntax error (línea 1 de tu PROOF): keyword ")" expected, but end-of-input (línea 21 de tu PROOF) was found
- gd_const_n2, ep. 2, paso 1, PROOF_FAILED: Outer syntax error (línea 2 de tu PROOF): keyword "]" expected, but keyword) (línea 10 de tu PROOF) was found
- gd_const_n2, ep. 2, paso 2, PROOF_FAILED: Undefined method: "nlinarith" (línea 1 de tu PROOF)
- gd_const_n2, ep. 2, paso 3, PROOF_FAILED: Undefined fact: "gd" (línea 1 de tu PROOF)
- gd_const_n2, ep. 2, paso 4, PROOF_FAILED: Failed to apply initial proof method (línea 1 de tu PROOF): goal (1 subgoal): 1. f (gd g [1, 1] x0) - f xs <= 1 / 10 *
- gd_const_n2, ep. 3, paso 1, PROOF_FAILED: Outer syntax error (línea 1 de tu PROOF): keyword "(" expected, but end-of-input (línea 1 de tu PROOF) was found
- gd_const_n2, ep. 3, paso 2, PROOF_FAILED: Failed to apply initial proof method: using this: f x0 + inner (g x0) (xs - x0) + 1 / 2 * (norm (g xs - g x0))2 <= f...
- gd_const_n2, ep. 3, paso 3, PROOF_FAILED: Outer syntax error (línea 98 de tu PROOF): command expected, but keyword [(línea 98 de tu PROOF) was found
- gd_const_n2, ep. 3, paso 4, PROOF_FAILED: Undefined method: "nlinarith" (línea 100 de tu PROOF)
- gd_const_textbook, ep. 1, paso 1, LEMMA_FAILED: Undefined method: "ring" (lema norm_sub_identity_aux, línea 5)
- gd_const_textbook, ep. 1, paso 2, LEMMA_FAILED: Undefined method: "nlinarith" (lema aux_norm_sub_identity, línea 16)
- gd_const_textbook, ep. 1, paso 3, LEMMA_FAILED: Failed to apply initial proof method: using this: f x + inner (g x) (y - x) + 1 / 2 * (norm (g y - g x))2 <= f y goal...
- gd_const_textbook, ep. 1, paso 4, LEMMA_FAILED: Undefined fact: "aux_norm_sub_identity" (lema gd_textbook_descent, línea 12)

- `gd_const_textbook`, ep. 2, paso 1, LEMMA_FAILED: Undefined fact: "inner_self_eq_norm" (lema aux_norm_sq_inner, línea 4)
- `gd_const_textbook`, ep. 2, paso 2, STATEMENT_DRIFT: el bloque de lemas no se admite: cadena sin cerrar
- `gd_const_textbook`, ep. 2, paso 3, LEMMA_FAILED: Undefined method: "ring" (lema textbook_norm_sub_identity, línea 9)
- `gd_const_textbook`, ep. 2, paso 4, LEMMA_FAILED: Undefined method: "nlinarith" (lema textbook_norm_sub_identity, línea 9)
- `gd_const_textbook`, ep. 3, paso 1, LEMMA_FAILED: Undefined method: "ring" (lema textbook_inner_norm_sub, línea 6)
- `gd_const_textbook`, ep. 3, paso 2, LEMMA_FAILED: Inner syntax error (lema textbook_norm_sub_identity, línea 2) at "2 = norm u 2 + norm v 2 - 2 * inner u v" Failed to...
- `gd_const_textbook`, ep. 3, paso 3, LEMMA_FAILED: Undefined fact: "norm_sq_eq_inner" (lema textbook_norm_sub_identity, línea 5)
- `gd_const_textbook`, ep. 3, paso 4, LEMMA_FAILED: Failed to apply initial proof method (lema textbook_norm_sub_identity, línea 5): goal (1 subgoal): 1. (norm (u - v))2...
- `gd_const_dt`, ep. 1, paso 1, PROOF_FAILED: Failed to apply terminal proof method (línea 1 de tu PROOF): goal (2 subgoals): 1. !!x0. f (gd g (replicate 0 1) x0)...
- `gd_const_dt`, ep. 1, paso 2, PROOF_FAILED: Undefined fact: "inner_le_norm" (línea 3 de tu PROOF)
- `gd_const_dt`, ep. 1, paso 3, PROOF_FAILED: Failed to apply terminal proof method (línea 2 de tu PROOF): goal (2 subgoals): 1. !!x0. f (gd g (replicate 0 1) x0)...
- `gd_const_dt`, ep. 1, paso 4, TIMEOUT: la compilación de la teoría final superó el límite de tiempo

6 Jerarquía de confianza entre jueces

Los tres jueces aceptan objetos distintos y dejan residuos de confianza distintos. Ordenados de menor a mayor independencia respecto del humano:

juez	qué certifica	en qué reposa la confianza
V_G	validez de una cadena en un grafo dado	el grafo, escrito por el profesor
V_P	una cota numérica de peor caso	el SDP, la certificación racional exacta
V_I	un teorema en el kernel de Isabelle/HOL	el kernel; la adecuación del triple congelado

Ningún juez elimina la confianza; cada uno la desplaza. V_G la desplaza al autor del grafo. V_P la desplaza a la formulación del PEP y a la implementación del certificador. V_I la desplaza a dos lugares que el kernel no puede inspeccionar: la correspondencia entre el triple de Hoare y el enunciado informal que se pretendía probar, y la matemática auxiliar importada. La Observación 2 muestra que incluso lo que el artículo de referencia reporta como "automático" contiene decisiones humanas en esa frontera. El objetivo del proyecto es medir ese residuo, no negarlo.

Residuo en esta corrida: la equivalencia entre la hipótesis `interp` y "f convexa y 1-suave con gradiente g" no está formalizada en Isabelle: es matemática de libro aceptada (Nesterov,

2018), como los lemas prestados de la bisección. También la adecuación del enunciado congelado a la cota que se pretende (la de Drori–Teboulle en los objetivos exactos) (Drori and Teboulle, 2014).

7 Entornos

sub.	método	objetivo de fase 2	juez	lemas	distr.	cad. mín.
01	bisección	$m_n \to c$, $f(c) = 0$, error $\leq (b-a)/2^{n+1}$	V_G, V_I	21	7	14
02	punto fijo	Banach en $[a, b]$: unicidad, convergencia, cotas	V_G	20	7	13
03	Taylor	resto de Lagrange; convergencia con derivadas acotadas	V_G	20	7	13
04	Newton	convergencia local cuadrática	V_G	18	7	11
05	secante	convergencia local de orden φ	V_G	21	8	13
06	pasos de GD	cota de peor caso $f(x_n) - f^*$ o $\|\nabla f(x_n)\|^2$ (PEP)	V_P	—	—	—
08	bisección	triple de Hoare de corrección total del programa `bisection`	V_I (programa)	—	—	—
09*	pasos de GD	4 teoremas de cota: `gd_const_dt`, `gd_const_n2`, `gd_const_textbook`, `gd_n1`	V_I (teorema)	—	—	—

Table 6: Entornos de `unirl`. En 01–05 los números se recalculan de los grafos (`proof_kb`): "lemas" cuenta todos los pasos, incluidos distractores y QED; "cad. mín." es el cierre de dependencias de QED. Objetivos de V_I disponibles: `bisection` (variante de programa) y 4 cotas de GD (variante de teorema). El punto fijo aún no tiene objetivo formal; Taylor, Newton y secante sólo disponen de juez de grafo. Proponentes: `gpt-6-luna` y `gpt-6-astra`, ambos con esfuerzo `high`, para todos los jueces (corridas en `runs/`: `gpt-6-luna`: 17, `gpt-6-astra`: 7). *: filas de este informe.

8 Experimentos realizados y previstos

1. *Este informe*: `gd_n1` con `gpt-6-luna` (high) (`20260924-062122-gpt-6-luna-high`; terminada; 3 episodios completos). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: PROOF_FAILED 8, INVALID_PROPOSAL 4.

2. *Este informe*: `gd_const_n2` con `gpt-6-luna` (high) (`20260924-073842-gpt-6-luna-high`; terminada; 3 episodios completos). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: PROOF_FAILED 12.

3. *Este informe*: `gd_const_textbook` con `gpt-6-luna` (high) (`20260924-082858-gpt-6-luna-high`; terminada; 3 episodios completos). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: LEMMA_FAILED 11, STATEMENT_DRIFT 1.

4. *Este informe*: `gd_const_dt` con `gpt-6-luna` (high) (`20260924-091710-gpt-6-luna-high`; terminada; 3 episodios completos). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: LEMMA_FAILED 5, PROOF_FAILED 4, TIMEOUT 3.

5. Relacionada: `pep_fval_mu001_n4` bajo V_P con `gpt-6-astra` (high) (`20260924-061156-gpt-6-astra-high`). 3 aceptados de 3 episodios completos, t^* mediana 1 ($B = 4$); etiquetas: CERTIFIED 3, INVALID_SCHEDULE 1; mejor $\tau = 0{,}0293248$ frente a $\tau_{\text{ref}} = 0{,}0295431$.

6. Relacionada: `pep_fval_mu001_n4` bajo V_P con `gpt-6-luna` (high) (`20260924-074351-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: NOT_IMPROVED 12; mejor $\tau = 0{,}0318308$ frente a $\tau_{\text{ref}} = 0{,}0295431$.

7. Relacionada: `pep_fval_mu01_n5` bajo V_P con `gpt-6-astra` (high) (`20260924-062058-gpt-6-astra-high`). 3 aceptados de 3 episodios completos, t^* mediana 1 ($B = 4$); etiquetas: CERTIFIED 3; mejor $\tau = 0{,}0075285$ frente a $\tau_{\text{ref}} = 0{,}0076055$.

8. Relacionada: `pep_fval_mu01_n5` bajo V_P con `gpt-6-luna` (high) (`20260924-075941-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: NOT_IMPROVED 12; mejor $\tau = 0{,}0087266$ frente a $\tau_{\text{ref}} = 0{,}0076055$.

9. Relacionada: `pep_gradnorm_n4` bajo V_P con `gpt-6-luna` (high) (`20260924-060113-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: NOT_IMPROVED 12; mejor $\tau = 0{,}0204082$ frente a $\tau_{\text{ref}} = 0{,}0169860$.

10. Relacionada: `pep_gradnorm_n4` bajo V_P con `gpt-6-astra` (high) (`20260924-060127-gpt-6-astra-high`). 3 aceptados de 3 episodios completos, t^* mediana 1 ($B = 4$); etiquetas: CERTIFIED 3; mejor $\tau = 0{,}0169094$ frente a $\tau_{\text{ref}} = 0{,}0169860$.

11. Relacionada: `pep_gradnorm_n4_neg` bajo V_P con `gpt-6-astra` (high) (`20260924-060745-gpt-6-astra-high`). 3 aceptados de 3 episodios completos, t^* mediana 1 ($B = 4$); etiquetas: CERTIFIED 3; mejor $\tau = 0{,}0169094$ frente a $\tau_{\text{ref}} = 0{,}0169860$.

12. Relacionada: `pep_gradnorm_n4_neg` bajo V_P con `gpt-6-luna` (high) (`20260924-072359-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: NOT_IMPROVED 11, INVALID_SCHEDULE 1; mejor $\tau = 0{,}0182615$ frente a $\tau_{\text{ref}} = 0{,}0169860$.

13. Relacionada: `pep_gradnorm_n6` bajo V_P con `gpt-6-astra` (high) (`20260924-060414-gpt-6-astra-high`). 3 aceptados de 3 episodios completos, t^* mediana 1 ($B = 4$); etiquetas: CERTIFIED 3; mejor $\tau = 0{,}0071538$ frente a $\tau_{\text{ref}} = 0{,}0072223$.

14. Relacionada: `pep_gradnorm_n6` bajo V_P con `gpt-6-luna` (high) (`20260924-061836-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: NOT_IMPROVED 12; mejor $\tau = 0{,}0124995$ frente a $\tau_{\text{ref}} = 0{,}0072223$.

15. Prevista: `gd_n1` con `gpt-6-astra` (`high`), misma configuración; aún sin corrida en `runs/`.

16. Prevista: `gd_const_n2` con `gpt-6-astra` (`high`), misma configuración; aún sin corrida en `runs/`.

17. Prevista: `gd_const_textbook` con `gpt-6-astra` (`high`), misma configuración; aún sin corrida en `runs/`.

18. Prevista: `gd_const_dt` con `gpt-6-astra` (`high`), misma configuración; aún sin corrida en `runs/`.

19. Prevista: transferencia entre objetivos (métrica 4), no implementada.

Observación 3 (Interpretación)**.** *Esta interpretación la redactó Claude a partir de los registros de las corridas; no es del autor.*

Luna no aceptó ninguno de los 12 episodios de los cuatro objetivos (48 intentos). No es que el juez sea inalcanzable: para los dos objetivos con n fijo, la demostración generada desde el certificado exacto del PEP es ACCEPTED *en unos 5 s (Observación 2). Lo que falta es escribirla. En `gd_n1`, 4 de los 12 intentos se perdieron por el analizador de respuestas anterior (nota de la*

carpeta); los otros 8 son demostraciones rechazadas. En `gd_const_textbook`*, el único* STATEMENT_DRIFT *es una comilla sin cerrar en el bloque de lemas: el filtro trata todo texto que no puede leer como un intento de salir del hueco, pero aquí se trata de un error de sintaxis, no de un intento de cambiar el enunciado. Los tres* TIMEOUT *de* `gd_const_dt` *son compilaciones que pasaron de 300 s.*

Los errores repiten el patrón de la bisección: métodos de Lean y Mathlib que Isabelle no tiene (`nlinarith` *6 veces,* `ring` *3,* `positivity` *1) y nombres de hechos inexistentes, casi todos con forma de nombres de Mathlib (*`sq_nonneg`*,* `inner_self_eq_norm`*,* `inner_le_norm`*); el hecho que Luna buscaba en Isabelle se llama* `power2_norm_eq_inner`*.*

Hay un resultado positivo. En la cota de Drori y Teboulle, Luna demostró e Isabelle verificó dos lemas propios: el desdoblamiento de la iteración,

$$\text{gd } g \text{ (replicate } (n+1)\ 1)\ x = \text{gd } g \text{ (replicate } n\ 1)\ (x - g(x)),$$

y la co-coercividad, $\langle g(x) - g(y), x - y\rangle \geq \|g(x) - g(y)\|^2$*, obtenida sumando las dos instancias de* `interp` *con una demostración Isar estructurada. La co-coercividad es un paso estándar de las demostraciones de libro de las cotas del descenso de gradiente, y es el primer lema verificado en las corridas de stepsize (en la bisección, los lemas verificados de Luna eran hechos auxiliares sobre logaritmos y techos). No bastó: la cota exacta para todo n necesita la combinación completa de desigualdades con los multiplicadores de Drori y Teboulle.*

Aquí el juez cuesta unos 5 s por intento y el modelo unos 3 min: a diferencia de la bisección, el cuello de botella es el proponente.

References

Bryant, D., Huerta y Munive, J. J., and Foster, S. (2026). Verifying numerical methods with Isabelle/HOL. v3, 21 Sep 2026; código: github.com/isabelle-utp/Verified_Numerical_Methods.

Das Gupta, S., Van Parys, B. P. G., and Ryu, E. K. (2024). Branch-and-bound performance estimation programming: A unified methodology for constructing optimal optimization methods. Mathematical Programming.

Drori, Y. and Teboulle, M. (2014). Performance of first-order methods for smooth convex minimization: A novel approach. *Mathematical Programming*, 145(1–2):451–482.

Grimmer, B., Shu, K., and Wang, A. L. (2024). Composing optimized stepsize schedules for gradient descent.

Nesterov, Y. (2018). *Lectures on Convex Optimization.* Springer.

Taylor, A. B., Hendrickx, J. M., and Glineur, F. (2017). Smooth strongly convex interpolation and exact worst-case performance of first-order methods. *Mathematical Programming*, 161(1–2):307–345.